第4章 第6节 多轮对话与上下文管理实战

- 来源: https://class.imooc.com/lesson/5269#mid=104907
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第4周 理解大模型开发
- 模块: 小试牛刀，做一个简单的翻译器
- 课节: 图文 1-2 多轮对话与上下文管理实战

1-2 多轮对话与上下文管理实战

第4章 第6节 多轮对话与上下文管理实战

> [!TIP] 阅读指南 前两节完成了API调用的基础入门。本节进入实战——多轮对话、上下文管理策略、response解析，最终做一个实用的智能翻译器。

6.1 大模型API的独特之处：用自然语言"设置参数"

从上面的例子可以看到，message其实是在"设置参数"，但与传统API有本质区别。

传统API的方式：必须传入精确的、预定义的参数值。

精确的、预定义的参数值

```
# 例如：调用天气API
response = requests.get(
 "https://api.weather.com/forecast",
 params={
     "city": "shanghai",          # 必须是准确的城市代码
     "units": "metric",           # 只能是 "metric" 或 "imperial"
     "format": "json"             # 只能是 "json" 或 "xml"
 }
)
# 如果你写 "units": "摄氏度"，API会直接报错

1
2
3
4
5
6
7
8
9
10

预览
```

大模型 API参数：可以用自然语言模糊地表达。

自然语言模糊地表达

```
# 同样是设置"参数"，但用的是自然语言
messages = [
 {
     "role": "system",
     "content": "你是专业的Python导师，擅长用简单的语言解释复杂概念。回答时要分步骤，配合代码示例。"
     # 这些都是模糊的描述，没有固定格式，但AI能理解
 }
]
# 你也可以写成："你是一个Python老师，请用通俗易懂的方式讲解，最好有代码例子"
# AI依然能理解并执行，虽然表达方式不同
```

这是AI API的革命性之处：你不需要记忆复杂的参数名和枚举值，只需要用日常语言说清楚你的需求。AI会自动理解并适应你的要求。

自然语言开发一定是未来的趋势。

6.2 多轮对话示例

你可能只看概念还不能很好的理解三种角色的作用，没关系，来看一个实例。

完整示例：质数主题的多轮对话

第一步:第一次问答

```
# 构造第1轮的messages
messages = [
    {"role": "system", "content": "你是数学老师,讲解要通俗易懂,必要时给出代码示例。"},
    {"role": "user", "content": "什么是质数?"}
]

# 调用API
response = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=messages
)

# 获取AI的回复
ai_response_1 = response.choices[0].message.content
print("AI:", ai_response_1)
# 输出示例: "质数是只能被1和自身整除的大于1的自然数。例如2、3、5、7都是质数。"
```

第二步:第二次问答

```
# 将上一轮的AI回复加入messages,继续追问
messages = [
    {"role": "system", "content": "你是数学老师,讲解要通俗易懂,必要时给出代码示例。"},
    {"role": "user", "content": "什么是质数?"},
    {"role": "assistant", "content": ai_response_1},  # 上一轮的AI回复，AI的回复角色应该标注为：assistant
    {"role": "user", "content": "10以内有哪些质数?"} # 这是第二次提问的问题
]

# 调用API
response = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=messages
)

# 获取AI的回复
ai_response_2 = response.choices[0].message.content
print("AI:", ai_response_2)
# 输出示例："10以内的质数有:2、3、5、7,一共4个。"
```

第三步:第三次问答

```
# 继续追问,携带完整的对话历史
messages = [
    {"role": "system", "content": "你是数学老师,讲解要通俗易懂,必要时给出代码示例。"},
    {"role": "user", "content": "什么是质数?"},
    {"role": "assistant", "content": ai_response_1},  # 第1轮AI回复
    {"role": "user", "content": "10以内有哪些质数?"},
    {"role": "assistant", "content": ai_response_2},  # 第2轮AI回复
    {"role": "user", "content": "请给出一个判断质数的Python函数,并说明时间复杂度。"} #第3轮
问题
]

# 调用API
response = client.chat.completions.create(
    model="qwen3.6-plus",
    messages=messages
)

# 获取AI的回复
ai_response_3 = response.choices[0].message.content
print("AI:", ai_response_3)
# 输出示例：AI会给出完整的质数判断函数和复杂度分析
```

> [!NOTE] 每一轮都要携带完整的对话历史(user 和 assistant 消息) system 消息保持不变，只设置一次，作为稳定的角色设定 messages 数组随着对话推进越来越长

三种角色的使用频率总结：

角色	使用频率	典型用法	备注
system	每次调用0-1条（开头）	设定角色、规则、输出格式	可选但推荐
user	至少1条，可多条	用户的真实输入	必需
assistant	0或多条	记录AI的历史回复	多轮对话必需

6.3 上下文管理策略

问题：对话越来越长，怎么办？

想象一个场景：和AI聊了30轮，从质数聊到算法，从排序聊到数据结构。如果每次都把完整的30轮对话历史塞进messages，会发生什么？

- 长度限制：模型有上下文窗口上限（如128K tokens），超出会被截断或直接报错
- 成本飙升：token数量直接影响费用，30轮对话可能消耗数万tokens

成本飙升

- 响应变慢：上下文越长，模型处理时间越长，延迟明显增加

响应变慢

> 核心矛盾是大模型需要历史才能理解上下文，但历史太长又会带来成本和性能问题。

3种实用策略

策略1 近期优先

保留最近3-5轮的完整对话，删除更早的轮次。

```
# 只保留最近3轮（伪代码）
recent_messages = messages[-6:]  # 每轮有user+assistant，所以最近3轮是取最后的6条
messages = [
    {"role": "system", "content": "你是Python导师..."},
    recent_messages
]
```

策略2 历史摘要

将早期对话压缩为要点，用一条assistant或system消息概括。

```
# 假设原本有5轮对话历史：
# 第1轮：user: "什么是递归？" -> assistant: "递归是函数调用自身的编程技巧...（200字）"
# 第2轮：user: "递归的终止条件是什么？" -> assistant: "终止条件是递归必须设置的...（150字）"
# 第3轮：user: "什么是调用栈？" -> assistant: "调用栈是程序执行时...（180字）"
# 如果全部保留，这3轮就占用了530字，token消耗大

# 压缩后的messages（将前3轮压缩为1条摘要）：
messages = [
    {"role": "system", "content": "你是Python导师，讲解要通俗易懂。"},
    # 早期3轮对话压缩为1条摘要（530字 -> 30字）
    {"role": "assistant", "content": "前面我们讨论了递归的基本概念、调用栈原理和终止条件设计。"},
    # 保留最近2轮完整对话
    {"role": "user", "content": "递归的时间复杂度如何分析？"},
    {"role": "assistant", "content": "可以通过递推关系式建立方程，常见的有主定理、递归树等方法。"},
    {"role": "user", "content": "能用斐波那契数列举个例子吗？"}
]
```

策略3 规则前置

将稳定不变的规则（身份、输出格式、禁止项）固定在system中，避免每轮在user消息中重复强调。

```
# 不好的做法：每轮重复
messages = [
    {"role": "user", "content": "你是Python导师，用通俗语言解释什么是递归？"},
    {"role": "assistant", "content": "..."},
   # 重复了身份设定、风格等固定的要求
    {"role": "user", "content": "你是Python导师，用通俗语言解释递归的时间复杂度？"}
]

# 好的做法：规则前置到system
messages = [
    {"role": "system", "content": "你是Python导师，讲解要通俗易懂，配合代码示例。"}, # 将"通俗易懂"放在system里，不用每次重复
    {"role": "user", "content": "什么是递归？"},
    {"role": "assistant", "content": "..."},
    {"role": "user", "content": "递归的时间复杂度如何分析？"}  # 简洁，只包含纯粹的提问
]
```

6.4 上下文管理策略实战

> [!TIP] 为了节省篇幅，也避免大家看完整代码看得头疼，我们只给出核心思路和关键代码。 完整源码参考：samples/chapter4/context_manager.py

用AI压缩历史对话：

```
# 假设有5轮历史对话，只保留最近2轮，前3轮需要压缩
full_history = [
    ("什么是递归？", "递归是函数调用自身的编程技巧，必须设置终止条件避免无限循环。..."),  # 约150字
    ("递归的终止条件是什么？", "终止条件是递归必须设置的基准情形，当满足条件时直接返回结果不再递归。例如计..."),  # 约120字
    ("什么是调用栈？", "调用度限制是1000层..."),  # 约130字
    ("递归的时间复杂度如何分析？", "可以通过递推关系式建立方程，常见的有主定理..."),
    ("能用斐波那契数列举个例子吗？", "斐波那契数列的递归实现：fib(n) = fib(n-1) + fib(n-2)...")
]

# 步骤1：提取需要压缩的早期对话
early_history = full_history[:3]  # 前3轮

# 步骤2：让AI生成摘要（核心压缩逻辑）
compress_messages = [
    {"role": "system", "content": "你是对话摘要助手，将多轮对话压缩为一句话总结。"},
    {"role": "user", "content": f"请将以下对话压缩为一句话摘要：\n\n{early_history}"}
]

summary = call_ai(compress_messages)
print(f"AI生成的摘要：{summary}")
# 输出示例："前面我们讨论了递归的基本概念、调用栈原理和终止条件设计。"（约30字）

# 步骤3：重组messages（摘要 + 最近2轮 + 当前问题）
recent_history = full_history[-2:]  # 最近2轮

optimized_messages = [
    {"role": "system", "content": "你是Python导师，讲解要通俗易懂。"},
    {"role": "assistant", "content": summary},  # 早期3轮压缩为1条摘要
    {"role": "user", "content": recent_history[0][0]},
    {"role": "assistant", "content": recent_history[0][1]},
    {"role": "user", "content": recent_history[1][0]},
    {"role": "assistant", "content": recent_history[1][1]},
    {"role": "user", "content": "斐波那契的时间复杂度是多少？"}  # 当前问题
]

# 压缩效果对比：
# 优化前：10条消息，约520字
# 优化后：7条消息，约170字
# 节省：350字（约67%）
```

上述代码的思路很简单，压缩早期对话这个活儿，也让AI来干。这可能比我们自己来压缩要更好。

> [!TIP] 经验法则 短对话（<5轮）：全部保留 中等对话（5-10轮）：保留最近5轮 + 早期摘要 长对话（>10轮）：保留最近3轮 + 精简摘要 + 相关性过滤 当然实际情况会更复杂，我们要灵活根据当前的情况来做出决策

6.5 返回结果response

实际上，LLM API的返回结果response对象包含的信息远不止文本。让我们打印完整结果：

```
print(response)


1
```

输出（简化版）：

```
ChatCompletion(
    id='chatcmpl-0c540cdb',  # 请求ID，用于追踪
    model='qwen3.6-plus',  # 使用的模型
    created=1763467746,  # 创建时间戳

    choices=[
        Choice(
            index=0,  # 候选序号
            message=ChatCompletionMessage(
                role='assistant',  # AI的角色
                content='你好！我是Qwen，是阿里巴巴集团旗下的通义实验室自主研发的超大规模语
言模型...'  # AI的回复文本
            ),
            finish_reason='stop'  # 结束原因：stop表示正常结束
        )
    ],

    usage=CompletionUsage(
        prompt_tokens=12,  # 输入消耗的Token数
        completion_tokens=91,  # 输出消耗的Token数
        total_tokens=103  # 总Token数
    )
)
```

关键信息：

- choices[0].message.content：AI的回复文本
- usage.total_tokens：消耗的Token数（用于计算费用）

```
usage.total_tokens
```

- finish_reason：模型停止生成的原因（重要！帮助判断回复是否完整） stop：正常结束 - 模型认为回答已完成，这是最理想的状态 length：达到长度限制 - 回复被强制截断，内容可能不完整。此时需要： 增大max_tokens参数（如果允许） 或者提示用户"回答过长，请分段提问" content_filter：内容被过滤 - 触发了安全审核机制（敏感词、违规内容） 模型拒绝回答或返回空内容 需要调整问题或检查输入内容

实用技巧

在正式项目中，务必检查finish_reason，避免将不完整的回复展示给用户。

```
if response.choices[0].finish_reason == 'length':
    print("回复被截断，请尝试缩短问题或增加max_tokens参数")
elif response.choices[0].finish_reason == 'content_filter':
    print("内容触发安全过滤，请调整问题")
```

6.6 从"Hello AI"到实用工具

一次简单的问候已经成功了，但这还不够实用。让我们做一个稍微有用的工具：智能翻译器。

需求分析

我们希望实现：输入一段文本，指定目标语言，AI返回翻译结果。

代码实现

> [!TIP] 完整源码参考：samples/chapter4/translator.py

创建文件translator.py：

```
from openai import OpenAI

# 先从环境变量获取API密钥，如果没有则使用硬编码值
api_key = os.getenv("DASHSCOPE_API_KEY") or "sk-xxxxxxxxxxxx"

client = OpenAI(
    api_key=api_key,
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

def translate(text, target_language="英文"):
    """智能翻译函数"""

    # 调用API
    response = client.chat.completions.create(
        model="qwen3.6-plus",
        messages=[
            {
                "role": "user",
                "content": f"请将以下文本翻译成{target_language}：\n\n{text}"
            }
        ]
    )

    # 返回翻译结果
    return response.choices[0].message.content

# 测试
if __name__ == "__main__":
    # 交互式输入
    print("=== 智能翻译器 ===")
    text = input("请输入要翻译的文本：")

    # 翻译成三种语言
    print("\n正在翻译...\n")

    english = translate(text, "英文")
    print(f"英文：{english}")

    french = translate(text, "法语")
    print(f"法语：{french}")
```

```
    spanish = translate(text, "西班牙语")
    print(f"西班牙语: {spanish}")
```

运行结果：

```
=== 智能翻译器 ===
请输入要翻译的文本: 小舟从此逝,江海寄余生

正在翻译...

英文: From now on, I'll drift away in my little boat,
Entrusting the rest of my life to rivers and seas.

法语: La petite barque s'en va désormais,
Je confierai le reste de ma vie aux fleuves et aux mers.

西班牙语: La pequeña barca se aleja ya para siempre;
en ríos y mares pasaré el resto de mi vida.
```

> [!IMPORTANT] 关于API密钥的安全管理 在示例代码中，会看到这样的代码： API_key = os.getenv("DASHSCOPE_API_KEY") or "sk-xxxxxxxxxxx" 1 代码块 这行代码的意思是： 先尝试从系统环境变量 DASHSCOPE_API_KEY 读取API密钥 如果环境变量不存在，则使用硬编码的占位符值 这样既保证了安全性，又方便快速测试 如何设置环境变量（Windows系统）： 打开"系统属性" → “高级” → “环境变量” 在"用户变量"中点击"新建" 变量名：DASHSCOPE_API_KEY 变量值：粘贴你的真实API密钥 点击"确定"保存，重启终端后生效 强烈建议：其实没必要这么麻烦。让Qoder帮忙设置环境变量，会根据操作系统（Mac/Windows/Linux）自动处理，避免手动配置的麻烦。 在真实项目里，绝对不要将API_KEY硬编码在代码里，而应该使用环境变量。

进阶 使用system角色优化

上面的代码可以工作，但还不够专业。我们可以用system角色来定义AI的行为，让同一句话翻译出不同的风格。

> [!TIP] 完整源码参考：samples/chapter4/translator_pro.py

改进版代码：

```
from openai import OpenAI

api_key = os.getenv("DASHSCOPE_API_KEY") or "sk-xxxxxxxxxxxx" # 替换成你的API密钥
client = OpenAI(
    api_key=api_key,
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)

def translate_pro(text, style="日常"):
    """专业翻译函数 - 支持风格控制"""

    # 使用system角色定义AI的行为
    messages = [
        {
            "role": "system",
            "content": f"""你是专业翻译，负责将中文翻译成英文。

翻译要求：
- 风格：{style}
- 保持原文的语气和情感
- 符合目标语言的表达习惯
- 只返回翻译结果，不要添加任何解释"""
        },
        {
            "role": "user",
            "content": text
        }
    ]

    response = client.chat.completions.create(
        model="qwen3.6-plus",
        messages=messages
    )

    return response.choices[0].message.content

# 测试不同风格
text = "小舟从此逝,江海寄余生"

print("文学风格：")
print(translate_pro(text, "文学风格，充满诗意"))
# 输出：From now on, I'll drift away in my little boat,
#       Entrusting the rest of my life to rivers and seas.
```

```
print("\n 口语风格：")
print(translate_pro(text, "口语化，简洁通俗"))
# 输出：I'll sail away now and spend the rest of my days wandering.
```

```
文学风格：
The little boat departs, never to return—
upon rivers and seas I'll spend my remaining years.

口语风格：
I'll drift away on this little boat, spending the rest of my life on rivers and seas.
```

这个示例展示了system角色的用处：同一句中文，通过调整system中的风格描述，可以得到完全不同的译文——文学风格保留了诗意和韵律，口语风格更加简洁直白

6.7 下一节预告

现在你已经掌握了API调用的基本方法。但在实际开发中，可能会遇到这样的问题：

> 翻译长文章时等待时间太长？如何控制输出长度？如何让翻译更直译或更意译？

下一节，我们将深入 API参数——通过stream、max_tokens、temperature等参数，精确控制AI的输出行为。

6.8 ■ 学点英语

中文	English	音标	说明
多轮对话	Multi-turn Dialogue	/ˈmʌlti tɜːrn ˈdaɪəlɑːg/	用户与AI之间连续多轮交互的对话形式
上下文管理	Context Management	/ˈkɑːntekst ˈmænɪdʒmənt/	控制对话历史长度和内容的策略
历史摘要	History Summarization	/ˈhɪstəri ˌsʌməraɪˈzeɪʃn/	将早期对话压缩为简洁摘要的上下文管理策略
规则前置	Rule Pre-positioning	/ruːl ˌpriːpəˈzɪʃənɪŋ/	将不变规则固定在system角色中避免重复
结束原因	Finish Reason	/ˈfɪnɪʃ ˈriːzn/	指示模型停止生成原因的状态字段